Group-Specific Responsibilities

Operational Scope for Groups 1-8

Module	Software Architecture and Process Models
Student Level	Level 2
Assignment Duration	7 weeks
Document	04 of 07
Groups 1-4	Apartment Management System
Groups 5-8	University Services Management Platform

1. Common Responsibilities for Every Team

- Appoint a Team Lead who represents the team in weekly cross-team synchronization.
- Maintain one Jira project with epics, user stories, acceptance criteria, tasks, bugs, sprint evidence, progress reports, and dependencies.
- Develop and maintain the microservices and frontend feature modules allocated to the team.
- Own the database or schema for each allocated microservice and provide versioned database scripts or migrations.
- Publish and maintain OpenAPI specifications and communicate breaking API changes before implementation.
- Use pull requests and code review; do not commit directly to protected main branches.
- Provide Postman tests, test execution evidence, and defect records for owned features.
- Participate in shared frontend integration, API Gateway routing, Docker Compose integration, deployment, and release validation.
- Submit team documentation and individual contribution evidence.

2. Suggested Role Allocation

Member	Suggested Primary Role	Mandatory Secondary Contribution
1	Scrum Master	Jira administration, progress evidence, risks, and one technical or testing task.
2	BA / Product Manager	SRS, acceptance criteria, traceability, and one frontend/backend/testing task.
3	Team Lead	API contracts, ADRs, integration, and implementation or automated testing.
4	UI/UX and Frontend Developer	Design consistency, shared frontend reviews, and feature implementation.
5	Frontend Developer	Feature module, Redux/API integration, validation, and frontend tests.
6	Backend Developer 1	Primary microservice implementation, persistence, validation, and unit tests.
7	Backend Developer 2	Second microservice implementation, integration, API docs, and unit tests.
8	QA / DevOps	Test coordination, defects, CI, Docker, deployment, and release checks.

This is a recommended allocation, not a restriction. Teams may combine or rotate roles, but role changes must be visible in Jira or progress reports and no student may be assigned only a ceremonial role.

3. Group 1 - Apartment Identity and Resident Management

Owned services	identity-access-service; resident-management-service
Owned frontend routes	/auth, /profile, /admin/users, /admin/roles, /residents, /owners, /tenants, /staff
Indicative entities	User, Role, Permission, ResidentProfile, OwnerProfile, StaffProfile, AccountStatus

Minimum user stories

- As an administrator, I can create and manage role-based user accounts.
- As a user, I can log in securely and access only permitted functions.
- As a manager, I can manage resident, owner, tenant, and staff profiles.
- As another service, I can validate a user or relationship through a documented API.

Cross-team dependency responsibility

Consumes unit/occupancy validation from Group 2. Provides identity, role, and user relationship validation to Groups 2-4.

Minimum definition of done

[ ] JWT authentication and protected endpoints work through the API Gateway.
[ ] Role restrictions are demonstrated with at least three distinct roles.
[ ] Resident/owner/tenant profiles can be created, updated, searched, and deactivated.
[ ] Other teams can validate users through stable APIs.

4. Group 2 - Apartment Property, Lease, and Occupancy

Owned services	property-unit-service; lease-occupancy-service
Owned frontend routes	/buildings, /floors, /units, /ownerships, /leases, /occupancies
Indicative entities	Building, Floor, Unit, UnitType, Ownership, Lease, Occupant, OccupancyStatus

Minimum user stories

- As a manager, I can create buildings, floors, units, and unit types.
- As a manager, I can record ownership and occupancy relationships.
- As a manager, I can create and activate a valid lease without date conflicts.
- As another service, I can confirm unit, owner, tenant, and occupancy status.

Cross-team dependency responsibility

Consumes user relationship validation from Group 1. Provides unit and occupancy validation to Groups 1, 3, and 4.

Minimum definition of done

[ ] Unit and lease business rules prevent invalid or overlapping active records.
[ ] Availability and occupancy history are visible in the frontend.
[ ] Cross-service validation APIs are documented and stable.
[ ] Status transitions are tested.

5. Group 3 - Apartment Billing, Utilities, and Payments

Owned services	billing-payment-service; utility-charge-service
Owned frontend routes	/charges, /invoices, /payments, /receipts, /utilities, /finance-dashboard
Indicative entities	ChargeRule, Invoice, InvoiceLine, Payment, Receipt, UtilityCharge, Adjustment

Minimum user stories

- As a finance officer, I can configure charges and generate invoices for eligible units.
- As a resident or owner, I can view invoices, outstanding balances, and receipts.
- As a finance officer, I can record a simulated payment and update invoice status.
- As management, I can review arrears and collection summaries.

Cross-team dependency responsibility

Consumes identity and role validation from Group 1 and unit/occupancy data from Group 2. Provides payment and balance status to relevant dashboard and notification features.

Minimum definition of done

[ ] Invoices preserve line-item values and calculate balances correctly.
[ ] Simulated payments create auditable records and receipts.
[ ] Invalid overpayment or unauthorized access is prevented.
[ ] Financial summaries reconcile with stored invoice and payment records.

6. Group 4 - Apartment Operations and Community

Owned services	operations-service; community-service
Owned frontend routes	/maintenance, /work-orders, /complaints, /facilities, /bookings, /visitors, /announcements, /notifications
Indicative entities	MaintenanceRequest, WorkOrder, Assignment, Facility, Booking, Visitor, Announcement, Notification

Minimum user stories

- As a resident, I can submit and track a maintenance request for my valid unit.
- As a coordinator, I can assign work and monitor technician progress.
- As a resident, I can reserve an available facility without a time conflict.
- As authorized staff, I can manage visitor records and publish announcements.

Cross-team dependency responsibility

Consumes identity and resident validation from Group 1 and unit status from Group 2. May consume balance status from Group 3 only when a documented booking rule requires it.

Minimum definition of done

[ ] Maintenance request to work-order closure is demonstrated end to end.
[ ] Facility conflict and eligibility rules are enforced.
[ ] Visitor and announcement access is role-restricted.
[ ] In-application notifications appear for selected workflow events.

7. Group 5 - University Identity and Directory

Owned services	identity-access-service; university-directory-service
Owned frontend routes	/auth, /profile, /users, /roles, /faculties, /departments, /service-units
Indicative entities	User, Role, Permission, Faculty, Department, ServiceUnit, UserAffiliation

Minimum user stories

- As an administrator, I can manage synthetic student and staff accounts and roles.
- As a user, I can log in and see a role-relevant dashboard.
- As an administrator, I can maintain faculties, departments, and service units.
- As another service, I can validate users, roles, affiliations, and service responsibilities.

Cross-team dependency responsibility

Provides identity, role, department, and service-unit validation to Groups 6-8. Consumes no domain database from other teams.

Minimum definition of done

[ ] JWT authentication and protected endpoints work through the API Gateway.
[ ] At least four role types are demonstrated.
[ ] Directory and affiliation data are searchable and maintainable.
[ ] Dependent teams can validate eligibility and responsibility through documented APIs.

8. Group 6 - University Facilities and Reservations

Owned services	facility-resource-service; reservation-service
Owned frontend routes	/facilities, /resources, /availability, /reservations, /approvals, /resource-dashboard
Indicative entities	Facility, Resource, ResourceType, AvailabilityRule, Reservation, ReservationApproval, TimeSlot

Minimum user stories

- As an authorized manager, I can maintain facilities, resources, hours, capacity, and rules.
- As an eligible user, I can search availability and request a reservation.
- As a resource manager, I can approve, reject, or cancel a reservation with a reason.
- As management, I can review usage and reservation status summaries.

Cross-team dependency responsibility

Consumes user, role, and department validation from Group 5. Provides resource and reservation data to Groups 7 and 8 when workflows reference a facility or event venue.

Minimum definition of done

[ ] Conflict, operating-hour, capacity, and eligibility rules are enforced.
[ ] Search and calendar views are usable.
[ ] Approval and cancellation transitions are tested.
[ ] Cross-service facility validation APIs are documented.

9. Group 7 - University Service Requests and Work Orders

Owned services	service-request-service; work-order-service
Owned frontend routes	/service-requests, /triage, /work-orders, /assignments, /technician, /service-dashboard
Indicative entities	ServiceRequest, RequestCategory, Priority, WorkOrder, Assignment, ActionLog, Resolution

Minimum user stories

- As a student or staff member, I can report a service problem and track its status.
- As a service desk officer, I can triage, prioritize, assign, escalate, or reject requests.
- As a technician, I can update assigned work and record resolution details.
- As management, I can review request and work-order summaries.

Cross-team dependency responsibility

Consumes user, role, department, and service-unit validation from Group 5 and facility validation from Group 6 when a location/resource is referenced. Provides completion status for feedback in Group 8.

Minimum definition of done

[ ] Request submission through closure is demonstrated.
[ ] Protected status transitions and required reasons are enforced.
[ ] Technician assignments and action logs are traceable.
[ ] Requester visibility and service summaries are accurate.

10. Group 8 - University Events, Communications, and Feedback

Owned services	event-service; communication-feedback-service
Owned frontend routes	/events, /registrations, /announcements, /notifications, /feedback, /engagement-dashboard
Indicative entities	Event, EventEligibility, Registration, Announcement, AudienceRule, Notification, FeedbackForm, FeedbackResponse

Minimum user stories

- As an organizer, I can create and publish an event with capacity and eligibility rules.
- As an eligible user, I can register or cancel before the deadline.
- As authorized staff, I can publish targeted announcements.
- As a user, I can receive notifications and provide feedback for completed services or events.

Cross-team dependency responsibility

Consumes user/role/department data from Group 5, optional venue validation from Group 6, and completed service-request status from Group 7. Provides notifications to project workflows through documented APIs.

Minimum definition of done

[ ] Registration capacity, date, status, and eligibility rules are enforced.
[ ] Targeted announcements are visible only to intended audiences.
[ ] Notifications are generated for selected events and workflow changes.
[ ] Feedback is linked only to eligible completed activities and summarized for authorized users.

11. Shared Integration Responsibilities for All Eight Teams

Area	Required Team Action
Shared frontend	Contribute only through reviewed pull requests; maintain feature boundaries; use shared authentication state, navigation, error handling, and design conventions.
API Gateway	Register routes, document base paths, verify authentication behavior, and test unavailable-service responses.
API contracts	Publish OpenAPI definitions, request/response examples, status codes, and change notes before dependent implementation.
Data ownership	Never access another service database directly; obtain needed information through APIs or approved asynchronous events.
Docker Compose	Provide service image, environment variables, health checks, dependencies, ports, and startup instructions.

Area	Required Team Action
Integration testing	Create at least one test that verifies a workflow involving another team service.
Release	Support release candidate validation, defect correction, tagged final release, deployment, and demonstration.
Documentation	Maintain service README, setup, API, database, test, and troubleshooting information.

12. Weekly Liaison Checklist

[ ] Review unresolved dependencies and blocked Jira issues.
[ ] Review API or shared model changes since the previous meeting.
[ ] Confirm shared frontend integration status and upcoming merge risks.
[ ] Confirm Docker Compose and environment configuration compatibility.
[ ] Record integration defects, owners, target dates, and retest status.
[ ] Update shared architecture decisions and contract registry.
[ ] Prepare a concise status update for the full team.